ZOHO Round 1 Questions

Pointers
Strings
Loops
Complex and Nested Loops
Control Flows

1. Output of this program?
```cpp
#include <iostream>
using namespace std;
int main()
{
    int i, j, var = 'A';
    for (i = 5; i >= 1; i--) {
        for (j = 0; j < i; j++)
            printf("%c ", (var + j));
        printf("\n");
    }
    return 0;
}
```

Output:
```
A B C D E
A B C D
A B C
A B
A
```

2. Output of the program?
```c
#include <stdio.h>
void f(char**);
int main()
{
    char *argv[] = { "ab", "cd", "ef", "gh", "ij", "kl" };
    f(argv);
    return 0;
}
```

```c
void f(char **p)
{
    char *t;
    t = (p += sizeof(int))[-1];
    printf("%s\n", t);
}
```

Output:
```
gh
```

3. Output of the following program?
```c
#include <stdio.h>
void dynamic(int s, ...)
{
    printf("%d ", s);
}
int main()
{
    dynamic(2, 4, 6, 8);
    dynamic(3, 6, 9);
    return 0;
}
```

Output:
```
2 3
```

4. Predict the output?
```c
#include <stdio.h>
int main()
{
    void demo();
    void (*fun)();
    fun = demo;
    (*fun)();
    fun();
    return 0;
```

```
}
void demo()
{
   printf("program ");
}
```

Output:
```
program program
```

5. Explain: What is the output of the following program?
```c
#include <stdio.h>
int main()
{
  char str1[] = "ZohoInterview";
  char str2[] = {'t', 'e', 's', 't', 't', 'e', 's', 't', '1'};
  int n1 = sizeof(str1)/sizeof(str1[0]);
  int n2 = sizeof(str2)/sizeof(str2[0]);
  printf("n1 = %d, n2 = %d", n1, n2);
  return 0;
}
```

Output:
```
n1 = 14, n2 = 9
```

Explanation:
- `str1` is a null-terminated string, so its length is 14 (13 characters + 1 null terminator).
- `str2` is an array of characters without a null terminator, so its length is 9.

6. Explain: Predict the Output
```c
#include <stdio.h>
int main()
{
   char str[] = "Aptitude";
   printf("%s %s %s\n", &str[5], &5[str], str+5);
   printf("%c %c %c\n", *(str+6), str[6], 6[str]);
   return 0;
```

```
}
```

Output:
```
ude ude ude
d d d
```

Explanation:
- `&str[5]`, `&5[str]`, and `str+5` all point to the 6th character of `str`, which is `'u'`.
- `*(str+6)`, `str[6]`, and `6[str]` all access the 7th character of `str`, which is `'u'`.

7. Explain: Predict the output of the below program
```c
#include <stdio.h>
#define SIZE(arr) sizeof(arr) / sizeof(*arr)
void fun(int* arr, int n)
{
    int i;
    *arr += *(arr + n - 1) += 10;
}
void printArr(int* arr, int n)
{
    int i;
    for(i = 0; i < n; ++i)
        printf("%d ", arr[i]);
}
int main()
{
    int arr[] = {10, 20, 30};
    int size = SIZE(arr);
    fun(arr, size);
    printArr(arr, size);
    return 0;
}
```

Output:
```
50 20 40
```

Explanation:

- `*arr` (10) is increased by the value of `*(arr + n - 1)` (30) plus 10, resulting in 50.
- `*(arr + n - 1)` (30) is increased by 10, resulting in 40.
- The final array is `{50, 20, 40}`.

8. Explain: Print the output
```c
#include <stdio.h>
struct st
{
   int x;
   struct st next;
};
int main()
{
   struct st temp;
   temp.x = 10;
   temp.next = temp;
   printf("%d", temp.next.x);
   return 0;
}
```

Output:
- This code will not compile because `struct st` contains a field of its own type, which is not allowed in C.

9. Explain: Identify the output of the following
```c
union test
{
   int x;
   char arr[8];
   int y;
};
int main()
{
   printf("%d", sizeof(union test));
   return 0;
}
```

Output:
```
8
```

```

**Explanation:**
- The size of the union is determined by the size of its largest member. `arr[8]` is the largest member, occupying 8 bytes.

### 10. Output of the program
```c
#include <stdio.h>
char *c[] = {"GeksQuiz", "MCQ", "TEST", "QUIZ"};
char **cp[] = {c+3, c+2, c+1, c};
char ***cpp = cp;
int main()
{
    printf("%s ", **++cpp);
    printf("%s ", *--*++cpp+3);
    printf("%s ", *cpp[-2]+3);
    printf("%s ", cpp[-1][-1]+1);
    return 0;
}
```

Output:
```
TEST sQuiz Z CQ
```

Explanation:
- `**++cpp` points to `"TEST"`.
- `*--*++cpp+3` points to `"Z"`.
- `*cpp[-2]+3` points to `"ST"`.
- `cpp[-1][-1]+1` points to `"eksQuiz"`.
```
11. **Question 1:**
```cpp
#include <iostream>
using namespace std;

int a = 20;

int main() {
  int a = 10;
  std::cout << a << "" << ::a;
  return 0;
```

```
}
```
Output: 1020

12. **Question 2:**
```cpp
#include<iostream>

void func(int *b){
    *b = 1;
}

int main() {
    int *a;
    int n;
    a = &n;
    *a = 0;
    func(a);
    std::cout << *a << std::endl;
    return 0;
}
```
Output: 1

13. **Question 3:**
```cpp
#include <iostream>
using namespace std;

class Test {
    static int i;
    int j;
};

int Test::i;

int main() {
    cout << sizeof(Test);
    return 0;
}
```
Output: 4

14. **Question 4:**

```c
#include <stdio.h>

int main() {
   int i;
   char ch[] = {'z','o','h','o'};
   char *ptr, *str1;
   ptr = ch;
   str1 = ch;
   i = (*ptr-- + ++*str1) - 10;
   printf("%d", i);
   return 0;
}
```
Output: 236

15. **Question 5:**
```c
int main() {
    int a[10][10] = {{1,2},{3,4},{5,6},{7,8},{9,10}};
    int *p = a[3];
    int result = (*p + 2) * a[4][1] + (++*p) + (*p + 7);
    printf("%d", result);
    return 0;
}
```
Output: 113

16. **Question 6:**
```c
#include <stdio.h>

void main() {
    int n = 21;
    int out = 1;
    while(n > 3) {
        n /= 2;
        out *= (n / 2);
    }
    printf("%d", n * out);
}
```
Output: 20

17. **Question 7:**
```c
#include <stdio.h>

int main() {
    int n[] = {8,1,3,9,4};
    int j, *y = n;
    for(j = 0; j < 5; j++) {
        if(j % 2 == 0)
            *y++;
    }
    printf("%d", *y);
    return 0;
}
```
Output: 9

18. **Question 8:**
```c
#include <stdio.h>

int main() {
    int x = 10;
    int y = 20;
    if(!(x ^ y))
        printf("0");
    else
        printf("1");
    return 0;
}
```
Output: 1

19. **Question 9:**
```c
#include <stdio.h>

int main() {
    int a[10][10] = {{1,2},{3,4},{5,6},{7,8},{9,10}};
    int *p = a[3];
    int result = (*p + 2) * a[4][1] + (++*p) + (*p + 13);
    printf("%d", result);
    return 0;
}
```

```
**Output:** 119

20. **Question 10:**
```c
#include <stdio.h>

int calc(int x, int *py, int **ppz) {
   int y, z;
   **ppz += 1;
   z = **ppz;
   *py += **ppz;
   y = *py;
   x += *py;
   return x + y + z;
}

void main() {
   int c, *b, **a;
   c = 5;
   b = &c;
   a = &b;
   printf("%d", calc(c, b, a));
}
```
Output: 35

21
```c
switch(5)
{
   default:printf("five");
   case 1:printf("one");
}
```
Output: `fiveone`

Explanation: The `default` case gets executed since there's no matching case for `5`. After `default`, execution falls through to `case 1`, resulting in printing `fiveone`.

22
```c
if(101)
```
```

```c
    printf("hai");
else
    printf("all");
```
Output: `hai`

Explanation: `101` is a non-zero value, which is considered `true` in C. Therefore, `printf("hai");` is executed.

23
```c
for(i=0;i<5;i++);
printf("%d",i);
```
Output: `5`

Explanation: The semicolon after the `for` loop creates an empty loop body. After the loop, `i` is `5`, which gets printed.

24
```c
void main()
{
    int a=10,b=20,c=30;
    printf("%d%d");
}
```
Output: Undefined behavior.

Explanation: The `printf` function is missing the variables to be printed, leading to undefined behavior.

25
```c
main()
{
    int i=0;
    switch(i)
    {
        case 0: i++;
                printf("%d", i);
        case 1:
                printf("%d", i);
        case 2:
```

```
            printf("%d", i);
    }
}
```
Output: `111`

Explanation: `case 0` increments `i` to `1`, then falls through to `case 1` and `case 2`, printing `1`, `1`, `2` in sequence.

26
```c
#define SQR(x) (x*x)
main()
{
    int a, b=3;
    a = SQR(b+2);
    printf("%d", a);
}
```
Output: `11`

Explanation: `SQR(b+2)` expands to `(b+2*b+2)`, i.e., `3+2*3+2 = 3+6+2 = 11`.

27
```c
int main()
{
    int i;
    int arr[5] = {1};
    for(i = 0; i < 5; i++)
        printf("%d", i);
}
```
Output: `01234`

Explanation: The loop prints the values of `i` from `0` to `4`.

28
```c
main()
{
    int x=0;
    if(x=0)
        printf("value of x is zero");
```

```c
    else
        printf("value of x is not zero");
}
```
Output: `value of x is not zero`

Explanation: `if(x=0)` assigns `0` to `x`, which is `false`. Thus, `printf("value of x is not zero");` gets executed.

29
```c
main()
{
    int i=0;
    for(i=0; i<20; i++)
    {
        switch(i)
        {
            case 0: i+=5;
            case 1: i+=2;
            case 5: i+=5;
            default: i+=4;
                    break;
        }
        printf("%d", i);
    }
}
```
Output: `1621`

Explanation:
- `i = 0`: case 0 adds 5 → `i = 5` falls through, case 1 adds 2 → `i = 7`, case 5 adds 5 → `i = 12`, default adds 4 → `i = 16` then breaks.
- `i = 16`: default adds 4 → `i = 20` then breaks.

30
```c
main()
{
    int x=20,y=35;
    x = y++ + x++;
    y = ++y + ++x;
    printf("%d%d",x,y);
}
```

```
Output: `5729`

Explanation:
- `x = y++ + x++` → `x = 35 + 20` → `x = 55`, `y = 36`, `x = 21`
- `y = ++y + ++x` → `y = 37 + 22` → `y = 59`

### 31
```c
void main()
{
    printf("\n98");
    printf("\b76");
    printf("\r54");
}
```
Output: `\r54` (results depend on terminal).

Explanation: `\b` moves the cursor back, `\r` moves to the start of the line.

32
```c
int main()
{
    int x=3;
    if(x==2); x=0;
    if(x==3) x++;
    else
    printf("x=%d",x);
}
```
Output: `1`

Explanation: First `if` does nothing (semicolon); second `if` is false; `else` not executed.

33
```c
int main()
{
    printf(" \" hai %% all \" ");
}
```
Output: `" hai % all "`
```

Explanation: Escape sequences print double quotes and percent sign.

34
```c
int main()
{
    printf(5+"welcome");
}
```
Output: `me`

Explanation: Pointer arithmetic prints string starting from the 5th index.

35
```c
int main()
{
    printf("%c", 5["welcomeall"]);
}
```
Output: `m`

Explanation: Equivalent to `printf("%c", "welcomeall"[5]);`.

36
```c
int main()
{
    int x = printf("welcome");
    printf("%d", x);
}
```
Output: `welcome7`

Explanation: `printf("welcome")` returns 7 (number of characters printed).

37
```c
int main()
{
    printf("%d", printf("%d", 1234));
}
```
Output: `12344`

Explanation: Inner `printf` prints `1234` and returns 4, outer `printf` prints `4`.

38
```c
int main()
{
   int a[25] = {1,2,3,4,5};
   printf("%d", sizeof(a)/sizeof(int));
}
```
Output: `25`

Explanation: `sizeof(a)` gives total bytes, divided by `sizeof(int)` gives array length.

39
```c
void main()
{
   int b=12;
   switch(printf("%c%d", 't', b))
   {
      case 0: printf("Hello"); break;
      case 1: printf("Bye"); break;
      case 5: printf("Welcome"); break;
      default: printf("Hi");
   }
}
```
Output: `t12Hi`

Explanation: `printf("%c%d", 't', b)` prints `t12` and returns `3` (3 characters printed). `default` case executes.

ZOHO Round 2 Questions

1. Greater on right side
You are given an array Arr of size N. Replace every element with the next greatest element (greatest element on its right side) in the array. Also, since there is no element next to the last element, replace it with -1.
Input:
N = 6

Arr[] = {16, 17, 4, 3, 5, 2}
Output:
17 5 5 5 2 -1
Explanation: For 16 the greatest element
on its right is 17. For 17 it's 5.
For 4 it's 5. For 3 it's 5. For 5 it's 2.
For 2 it's -1(no element to its right).
So the answer is 17 5 5 5 2 -1
Link: https://www.geeksforgeeks.org/problems/greater-on-right-side4305/1

2. Given a Boolean matrix mat[M][N] of size M X N, modify it such that if a matrix cell mat[i][j] is 1 then make its adjacent cells as 0.
Input:
1 0 1
0 1 0
1 1 1
Output:
0 0 0
0 0 0
1 0 1
Explanation:
For the cell mat[0][0] which is 1, its adjacent cells (mat[0][1] and mat[1][0]) are modified to 0.
For the cell mat[1][1] which is 1, its adjacent cells (mat[0][1], mat[1][0], mat[1][2], and mat[2][1]) are modified to 0.
The modification is not applied to the cell mat[2][2] as it doesn't have all four adjacent cells.

3. Equilibrium index of an array is an index such that the sum of elements at lower indexes is equal to the sum of elements at higher indexes. For example, in an array A:
Example :
Input: A[] = {-7, 1, 5, 2, -4, 3, 0}
Output: 3
3 is an equilibrium index, because:
A[0] + A[1] + A[2] = A[4] + A[5] + A[6]
Input: A[] = {1, 2, 3}
Output: -1

4. In MS-Paint, when we take the brush to a pixel and click, the color of the region of that pixel is replaced with a new selected color. Following is the problem statement to do this task.
Given a 2D screen, location of a pixel in the screen and a color, replace color of the given pixel and all adjacent same colored pixels with the given color.
Example:
Input:
screen[M][N] = {{1, 1, 1, 1, 1, 1, 1, 1},
{1, 1, 1, 1, 1, 1, 0, 0},

{1, 0, 0, 1, 1, 0, 1, 1},
{1, 2, 2, 2, 2, 0, 1, 0},
{1, 1, 1, 2, 2, 0, 1, 0},
{1, 1, 1, 2, 2, 2, 2, 0},
{1, 1, 1, 1, 1, 2, 1, 1},
{1, 1, 1, 1, 1, 2, 2, 1},
};

x = 4, y = 4, newColor = 3

The values in the given 2D screen indicate colors of the pixels.
x and y are coordinates of the brush, newColor is the color that
should replace the previous color on screen[x][y] and all surrounding
pixels with same color.

Output:
Screen should be changed to following.

screen[M][N] = {{1, 1, 1, 1, 1, 1, 1, 1},
{1, 1, 1, 1, 1, 1, 0, 0},
{1, 0, 0, 1, 1, 0, 1, 1},
{1, 3, 3, 3, 3, 0, 1, 0},
{1, 1, 1, 3, 3, 0, 1, 0},
{1, 1, 1, 3, 3, 3, 3, 0},
{1, 1, 1, 1, 1, 3, 1, 1},
{1, 1, 1, 1, 1, 3, 3, 1},
};

5. Given a matrix of 2D array of n rows and m coloumns. Print this matrix in ZIG-ZAG fashion as shown in figure.

Example:
Input:
1 2 3
4 5 6
7 8 9
Output:
1 2 4 7 5 3 6 8 9

6. Remove the duplicates in the String.
Testcase 1:
Input: Java1234
Output: Javb1234 (Remove the second ‘a’ as it is duplicated)
Testcase 2:
Input: Python1223:
Output: Python1234 (Replace the second 2 with 3, and replace 3 with 4 as 3 is replaced for the duplicated 2)
Testcase 3:

Input: aBuzZ9900
Output: aBuzC9012
(Replace the second ‘Z’ with ‘C’ as ‘a’ and ‘B’ are already there in the String. Replace with capital C as the letter to be replaced is capital Z. The second 9 turns out to be zero and the zero turns out to ‘1’ and the second zero turns out to ‘2’)

7. Print whether the version is upgraded, downgraded or not changed according to the input given.
example: Input : Version1 4.8.2 Version2 4.8.4 Output: upgraded, Input : Version1 4.0.2 Version2 4.8.4 Output: downgraded

8. Print all possible subsets of the given array whose sum equal to given N.
example: Input: {1, 2, 3, 4, 5} N=6 Output: {1, 2, 3}, {1, 5}, {2, 4}

9. Reverse the words in the given String1 from the first occurrence of String2 in String1 by maintaining white Spaces.
example: String1 = Input: This is a test String only String2 = st Output: This is a only String test

10. calculate Maximum number of chocolates can eat and Number of wrappers left in hand.
Money: Total money one has to spend.
Price: price per chocolate.
wrappers: minimum number of wrappers for exchange choco: number of chocolate for wrappers.
Max visit: Maximum number of times one can visit the shop.(if zero consider it infinite)
example: input: Money:40 Price:1 wrappers:3 choco:1 Max visit:1 Output: total chocolate can eat: 53 wrappers left in hand:14

11. Sample Input-
2
Hacker
Rank
Sample Output-
Hce akr
Rn ak

13. Print the word with odd letters – PROGRAM
Sample Output-+4477
P P
 R R
  O O
   G
  R R
 A A
M M

14. Sample Input – Alternate Sorting
Input: {1, 2, 3, 4, 5, 6, 7}
output: {7, 1, 6, 2, 5, 3, 4}

15. Given an array of values persons[], each represents the weight of the persons. There will be infinite bikes available. Given a value K which represents the maximum weight that a bike accommodates. Along with that one more condition, a bike can carry two persons at a time. You need to find out the least number of times, the bike trips are made.

16. Assume there exists infinite grid, you're given initial position x, y. Inputs will be movements either L or R or U or D. After n inputs, you need to give the current position.

- Input:
- 4 5 //initial position x, y
- 9 //number of movements
- U L R R D D U L R //7 movements
- Output:

5 5

- Given a matrix NxN, you are initially in the 0, 0 position. The matrix is filled with ones and zeros. Value "one" represents the path is available, while "zero" represents the wall. You need to find the can you able to reach the (N-1)x(N-1) index in the matrix. You can move only along the right and down directions if there's "one" available.

- Input:
- 5 //N value
- 1 0 1 0 0
- 1 1 1 1 1
- 0 0 0 1 0
- 1 0 1 1 1
- 0 1 1 0 1
- Output:

Yes

17. Given an array of integers, compute the maximum value for each integer in the index, by either summing all the digits or multiplying all the digits. (Choose which operation gives the maximum value)

- Input:
- 5
- 120 24 71 10 59
- Output:
- 3 8 8 1 45

Explanation: For index 0, the integer is 120. Summing the digits will give 3, and whereas Multiplying the digits gives 0. Thus, maximum of this two is 3.

18. -1 represents ocean and 1 represents land find the number of islands in the given matrix.

Input: n*n matrix
 1 -1 -1 1
-1 1 -1 1
-1 -1 1 -1
-1 -1 -1 1
Output: 2 (two islands that I have
bold in matrix at 1, 1 and 2, 2)

19. Print all the possible subsets of array which adds up to give a sum.
Input: array{2, 3, 5, 8, 10}
 sum=10
Output: {2, 3, 5}
 {2, 8}
 {10}

20. There is a circular queue of processes. Every time there will be certain no of process skipped and a particular start position. Find the safe position.
Input: Number of process:5
 Start position:3
 Skip: 2nd
Output: 1 will be the safest position
(Logic: 1 2 3 4 5 starting from 3, 5th process will be skipped
 1 2 3 4 5 process 2 will be skipped
 1 2 3 4 5 process 4 will be skipped
 1 2 3 4 5 process 3 will be skipped, so safest process is 1.

21. Given N. print the following snake pattern (say N = 4). condition: must not use arrays (1D array or 2D array like Matrix).
1 2 3 4
8 7 6 5
9 10 11 12
16 15 14 13

22. Given N. print the Latin Matrix (say N = 3). condition: must not use strings(aka character literals), arrays (both 1D and 2D), inbuilt functions(like rotate).
A B C
B C A
C A B

23. Given a number N. find the minimum count of numbers in which N can be represented as a sum of numbers x1, x2, … xn. where xi is number whose digits are 0s and 1s.
example 1) i/p : N = 33
o/p : count = 3. 33(11 + 11 + 11)

some other possibilities of 33 is (11 + 11 + 10 + 1), (11 + 10 + 10 + 1 + 1), (10 + 10 + 10 + 1 + 1 + 1)

24. Finding all permutations of a string. (backtracking approach).

25. Given an array of integers, write a program to re-arrange the array in the given form. 1st_largest, 1st_smallest, 2nd_largest, 2nd_smallest, 3rd_largest ……. etc.

26. Sort the given elements in decending order based on the number of factors of each element – Solution 1

27. Find whether the given number is palindrome or not. Don't use arrays or strings

28. Reverse the given string keeping the position of special characters intact

29. Find the shortest path from one element to another element in a matrix using right and down moves alone. The attached solution uses moves in all directions. – Solution 4

30. Pattern

31. Pangram Checking
Check whether all english alphabets are present in the given sentence or not
I/P: abc defGhi JklmnOP QRStuv wxyz
O/P: True
I/P: abc defGhi JklmnOP QRStuv
O/P: False

32. Password Strength
Find the strength of the given password string based on the conditions
Four rules were given based on the type and no. of characters in the string.
Weak – only Rule 1 is satisfied or Rule 1 is not satisfied
Medium – Two rules are satisfied
Good – Three rules satisfied
Strong – All Four rules satisfied

I/P: Qw!1 O/P: Weak
I/P: Qwertyuiop O/P: Medium
I/P: QwertY123 O/P: Good
I/P: Qwerty@123 O/P: Strong

33. First Occurrences
Given two strings, find the first occurrence of all characters of second string in the first string and print the characters between the least and the highest index
I/P: ZOHOCORPORATION PORT

O/P: OHOCORPORAT
Explanation: The index of P in first string is 7, O is 1, R is 6 and T is 11. The largest range is 1 – 11.
So print the characters of the first string in this inex range i.e. OHOCORPORAT

34. Matrix Diagonal sum
Given a matrix print the largest of the sums of the two triangles split by diagonal from top right to bottom left
I/P:
3 3
1 2 3
4 5 6
7 8 9
O/P: 38

35. Matrix Addition
Given n integer arrays of different size, find the addititon of numbers represented by the arrays
I/P: 4
3 5 4 2
2 4 5
4 5 6 7 8
4 9 2 1
1 2
O/P: 50856

36. Many students will able to solve 3 problems in this round. So make sure you stand apart from the crowd.Their vacancy is going to be 5 for a team. The performance in this round could be taken as a tie breaker for round 3.
input : aaabbcc
output : abc

37. Evaluate the expression and sort and print the output. Getting the input is the tricky part
Input:
Number of input : 4
2*3
2^2^2
35
3*1
Output:
3*1
2*3
2^2^2
35

38. Given a 6 blocks, of different height h1, …, h6 . Make 2 towers using 3 Blocks for each tower in desired height h1, h2. Print the blocks to be used in ascending order
Input:
1 2 5 4 3 6
height of tower: 6 15
Output :
1 2 3 & 4 5 6

39. Given a 5X5 chess board as input. 9 knights are placed in the board. Print whether the configuration valid or Invalid.

40. Given a number, print all the code that can be formed with z={a=1, .., z=26}.
1123
{1, 1, 2, 3} = aabc
{11, 2, 3} = kbc
{1, 1, 23} = aaw
{11, 23} = kw

41. Given a String with or without special characters find if it is Palindrome or Not.. No splitting of array must be done or No additional spaces must be used for storing the array..
Eg: RACE CAR
Eg: I DID, DID I ?

42. Given an array of integers of size n. Convert the array in such a way that if next valid number is same as current number, double its value and replace the next number with 0. After the modification, rearrange the array such that all 0’s are shifted to the end.
Input : arr[] = {2, 2, 0, 4, 0, 8}
Output : 4 4 8 0 0 0
Input : arr[] = {0, 2, 2, 2, 0, 6, 6, 0, 0, 8}
Output : 4 2 12 8 0 0 0 0 0 0

43. TWISTED PRIME NUMBER
A number is said to be twisted prime if it is a prime number and reverse of the number is also a prime number.
Input : 97
Output : Twisted Prime Number
Explanation: 97 is a prime number
and its reverse 79 is also a prime
number.

44. Given an array A[] and a number x, check for pair in A[] with sum as x.
Eg : Input {1, 2, 4, 3, 5, 6}
SUM : 5
Output : 2 (1, 4) & (2, 3)

45. Largest Sum Contiguous Subarray
(Kadane' Algorithm)

46. Diamond pattern : for given input size -> Here 3

```
  *
 ***
*****
 ***
  *
```

46. Given a text and a wildcard pattern, implement wildcard pattern matching algorithm that finds if wildcard pattern is matched with text. The matching should cover the entire text (not partial text).
The wildcard pattern can include the characters '?' and '*'
'?' – matches any single character
'*' – Matches any sequence of characters (including the empty sequence)
Example:
Text = "baaabab",
Pattern = "*****ba*****ab",
output : true
Pattern = "baaa?ab", output : true
Pattern = "ba*a?", output : true
Pattern = "a*ab", output : false

47. Given an input string and a dictionary of words, find out if the input string can be segmented into a space-separated sequence of dictionary words. See following examples for more details.
Consider the following dictionary
{ i, like, sam, sung, samsung, mobile, ice,
cream, icecream, man, go, mango}

Input: ilike
Output: Yes
The string can be segmented as "i like".

Input: ilikesamsung
Output: Yes
The string can be segmented as "i like samsung"
or "i like sam sung".<>
48. Print the following pattern

```
   1
  3 2
 6 5 4
10 9 8 7
```

10 9 8 7
 6 5 4
  3 2
   1

49. Given an array as input, The condition is if the number is repeated you must add the number and put the next index value to 0. If the number is 0 print it at the last.
Eg: arr[] = { 0, 2, 2, 2, 0, 6, 6, 0, 8}
Output: 4 2 12 8 0 0 0 0 0 .

49. Given two Strings s1 and s2, remove all the characters from s1 which is present in s2.
Input: s1="expErIence", s2="En"
output: s1="exprIece"

50. Find the next greater element for each element in given array.
input: array[]={6, 3, 9, 10, 8, 2, 1, 15, 7};
output: {7, 5, 10, 15, 9, 3, 2, _, 8}
If we are solving this question using sorting, we need to use any O(nlogn) sorting algorithm.

51. Print all distinct permutations of a given string with duplicate characters.
https://www.geeksforgeeks.org/distinct-permutations-string-set-2

52. Given a number, find the next smallest palindrome.

53. Given an array with repeated numbers, Find the top three repeated numbers.
input: array[]={3, 4, 2, 3, 16, 3, 15, 16, 15, 15, 16, 2, 3}
output: 3, 16, 15

54. Given two dimensional matrix of integer and print the rectangle can be formed using given indices and also find the sum of the elements in the rectangle
Input: mat[M][N] = {{1, 2, 3, 4, 6}, {5, 3, 8, 1, 2}, {4, 6, 7, 5, 5}, {2, 4, 8, 9, 4} };
index = (2, 0) and (3, 4)
Output:
Rectangle
4 6 7 5 5
2 4 8 9 4
sum 54

55. Find the result subtraction, multiplication, division of two integers using + operator.
Input: 6 and 4
output:
addition 6+4 = 10, subtraction 6+(-4) = 2, multiplication = 24, division = 1
Input : -8 and -4
Output:

addition -8+(-4) = -12, subtraction (-8)+(-(-4)) = -4, multiplication = 32, division = 2

56. Given a sentence of string, in that remove the palindrome words and print the remaining.
Input:
He did a good deed
Output:
He good
Input:
Hari speaks malayalam
Output:
Hari speaks

57. Given two dates, find total number of days between them.

Input: dt1 = {10, 2, 2014} dt2 = {10, 3, 2015}
Output: 393
dt1 represents “10-Feb-2014” and dt2 represents “10-Mar-2015” The difference is 365 + 28
Input: dt1 = {10, 2, 2000} dt2 = {10, 3, 2000}
Output: 29
Note that 2000 is a leap year
Input: dt1 = {10, 2, 2000} dt2 = {10, 2, 2000}
Output: 0
Both dates are same
Input: dt1 = {1, 2, 2000}; dt2 = {1, 2, 2004};
Output: 1461
Number of days is 365*4 + 1

58. Let 1 represent ‘A’, 2 represents ‘B’, etc. Given a digit sequence, count the number of possible decodings of the given digit sequence.
Examples:
Input: digits[] = “121”
Output: 3 // The possible decodings are “ABA”, “AU”, “LA”
Input: digits[] = “1234” Output: 3
// The possible decodings are “ABCD”, “LCD”, “AWD”

59. Print all possible words from phone digits

60. Print longest sequence between same character

Ex I/p abccccbba
O/p 8 (from a to a)
I/p aaaaaaaa
O/p 6

61. sort the array odd numbers in ascending and even numbers in descending.
I/p 5 8 11 6 2 1 7
O/p 1 5 7 11 8 6 2

62. It's about anagram.i/p was array of strings .and a word was given to find whether it has anagram in given array.
I/p catch, got, tiger, mat, eat, Pat, tap, tea
Word: ate
O/p eat, tea

63. array of numbers were given to find a number which has same sum of numbers in it's either side.
I/p 1, 2, 3, 7, 6
O/p 7(has 1+ 2+3 in left 6 in right)

64. prime number – print n prime numbers

65. prime factor – sort the array based on the minimum factor they have.

66. adding a digit to all the digits of a number eg digit=4, number = 2875, o/p= 612119

67. form the largest possible number using the array of numbers.

68. lexicographic sorting.

69. given a set of numbers, and a digit in each iteration, if the digit exists in any of the numbers, remove its occurrences and ask for the next digit till the list becomes empty.

70. Check if a number 'a' is present in another number 'b.

71. Find the extra element and its index
Input : [10, 20, 30, 12, 5]
[10, 5, 30, 20]
Output : 12 is the extra element in array 1 at index 4

Input : [-1, 0, 3, 2]
[3, 4, 0, -1, 2]
Output : 4 is the extra element in array 3 at index 5

72. Find the least prime number that can be added with first array element that makes them divisible by second array elements at respective index (check for prime numbers under 1000, if exist return -1 as answer) & (Consider 1 as prime number)
Input : [20, 7]
[11, 5]

Output : [1, 3]

Explanation :
(20 + ?) % 11
(7 + ?) % 5

73. Sort the array elements in descending order according to their frequency of occurrence

Input : [2 2 3 4 5 12 2 3 3 3 12]
Output : 3 3 3 3 2 2 2 12 12 4 5
Explanation : 3 occurred 4 times, 2 occurred 3 times, 12 occurred 2 times, 4 occurred 1 time, 5 occurred 1 time

Input : [0 -1 2 1 0]
Output : 0 0 -1 1 2
Note : sort single occurrence elements in ascending order

74. Print true if second string is a substring of first string, else print false.
Note : * symbol can replace n number of characters
Input : Spoon Sp*n Output : TRUE
Zoho *o*o Output : TRUE
Man n* Output : FALSE
Subline line Output : TRUE

75. Print second frequently occurring number in given series
Example :
Input: 1 1 2 3 1 2 4
Output: 2
Explanation: 1 occurs 3 times, 2 occurs 2 times, 3 occurs 1 time and 4 occurs 1 time. Hence second frequently occurring number in given series is 2

76. Print only numbers which is present in Fibonacci series (0 1 1 2 3 5 8 ……..)
Example:

Input: 2 10 4 8
Output: 2 8
Input: 1 10 6 8 13 21
Output: 1 8 13 21

77. Print pattern like this
Example:
Input: 1
Output: 0

Input: 2
Output:
0 0
0 1
1 0
1 1

Input: 3
Output:
0 0 0
0 0 1
0 1 0
0 1 1
1 0 0
1 0 1
1 1 0
1 1 1

78. NxN matrix will be provided. 0->block, 1->Not a block
Always starting point is (0,0), Ending point is (N-1,N-1).
You have to go from starting point to ending point. One valid solution is enough.
Example:

Input:
N=4
1 1 0 0
1 0 0 1
1 1 1 1
0 0 0 1
Output:
_ 1 0 0
_ 0 0 1
_ _ _ _
0 0 0 _

79. Insert 0 after consecutive (K times) of 1 is found.
Example:
Input:
Number of bits: 12
Bits: 1 0 1 1 0 1 1 0 1 1 1 1
Consecutive K: 2

Output:

1 0 1 1 0 0 1 1 0 0 1 1 0 1 1 0

80. Find the maximum of three numbers?

81. Print the total number of odd and even digits in the given number.

Ex. Input : 1234567

Output : ODD 4
EVEN 3

82. Find the second maximum among the given numbers.
Ex. INPUT :

Size of Array : 8
Enter the elements : 2 5 1 6 2 6 7 10

OUTPUT :

7

Ex. INPUT :

Size of Array : 4
Enter the elements : 4 1 2 2

OUTPUT :

2

Ex. INPUT :

Size of Array : 1
Enter the elements : 1

OUTPUT :

No second maximum

83. Print the following pattern
Ex. INPUT : 5

OUTPUT :

```
        1
       1 1
      1 2 1
     1 3 3 1
    1 4 6 4 1
```

Ex. INPUT : 7

OUTPUT :

```
        1
       1 1
      1 2 1
     1 3 3 1
    1 4 6 4 1
   1 5 10 10 5 1
  1 6 15 20 15 6 1
```

84. Given a two dimensional array which consists of only 0's and 1's. Print the matrix without duplication.

Ex. INPUT :

Enter Row Size : 4
Enter column size : 3
Enter the matrix :
1 0 1
1 1 0
1 1 1
1 0 1

OUTPUT :

Unique Matrix :
1 0 1
1 1 0
1 1 1

85. Given an array of positive numbers. Print the numbers which have longest continuous range.

Ex. INPUT :

Enter array size : 8
Enter arryay elements : 1 3 10 7 9 2 4 6

OUTPUT :

1 2 3 4

Ex. INPUT :

Enter array size : 8
Enter arryay elements : 1 3 9 7 8 2 4 6

OUTPUT :

1 2 3 4
6 7 8 9

86. Given two arrays. Find its union.
Input :

Enter size of first array : 6
Enter the elements : 1 2 3 4 5 3
Enter size of second array : 4
Enter the elements : 1 2 7 5

OUTPUT :

1 2 3 4 5 7

87. Given an array of numbers. Print the numbers without duplication.

INPUT :

Enter the array size : 4
Enter the elements : 1 1 2 4

OUTPUT :

1 2 4

88. Given an array of numbers and a number k. Print the maximum possible k digit number which can be formed using given numbers.
INPUT :

Enter the array size : 4
Enter the elements : 1 4 973 97
Enter number of digits : 3

OUTPUT :

974

INPUT :

Enter the array size : 6
Enter the elements : 1 4 89 73 9 7
Enter number of digits : 5

OUTPUT :

98973

89. Given an array of numbers and a window of size k. Print the maximum of numbers inside the window for each step as the window moves from the beginning of the array.
INPUT :

Enter the array size : 8
Enter the elements : 1,3,5,2,1,8,6,9
Enter the window size : 3

OUTPUT :

5 5 5 8 8 9

90. Given a string, reverse only vowels in it; leaving rest of the string as it is.
Input : abcdef
Output : ebcdaf

91. Write a program to check if the given words are present in matrix given below. The words can be left to right, top to bottom and the diagonals (in top to bottom direction)

92. Write a program to form lines using given set of words. The line formation should follow below rules.
i. Total characters in a single line excluding the space between the words and the favorite character should not exceed the given number.
ii. Favorite character is case insensitive.
iii. Words should not be broken up. Complete words alone should be used in a single line. A word should be used in one line only.

Input : Max char per line = 10
Favorite character = 'o'

Words : Zoho, Eating, Watching, Pogo
Loving, Mango
Output : Watching Zoho
Eating Mango
Loving Pogo.

93. Adding 2 numbers
Given 2 huge numbers as separate digits, store them in array and process them and calculate the sum of 2 numbers and store the result in an array and print the sum.
Input:
Number of digits:12
9 2 8 1 3 5 6 7 3 1 1 6
Number of digits:9
7 8 4 6 2 1 9 9 7
Output :
9 2 8 9 2 0 2 9 5 1 1 3

94. Given sorted array check if two numbers sum in it is a given
value
Input
Array = {1 3 4 8 10 } N = 7
output
true

95. Compiuting value of sin (x)
Input x = 30 n = 10
output = 0.5
Hint : The equation sin(x) = x – x^3 / 3! + x^5 / 5! – ….

96. Write function to find multiplication of 2 numbers using +
operator You must use minimum possible iterations.
Input: 3 , 4
Output 12

97. Given array find maximum sum of contiguous sub array
{-2 -3 4 -1 -2 1 5 -3}
output 7 elements [4 -1 -2 1 5]

98. Given unsorted array find all combination of the element for a given sum. Order should be maintained.
Input :
8 3 4 7 9 N=7
Output
{3 4 } {7}

99. Given an odd length word which should be printed from the middle of the word.
The output should be in the following pattern.
Example:
Input: PROGRAM
Output:

```
      G
     GR
    GRA
   GRAM
  GRAMP
 GRAMPR
GRAMPRO
```

100. It is a program to implement Least Recently Used (LRU) concept. Given a key, if it is already existed then it should be marked as recently used otherwise a value should be stored which is given as input and marked as recently used. The capacity is to store only 10 key, value pairs. If the table is full and given a new key; the key, value pair which is not recently used should be deleted which gives feasibility to store the new key, value pair.

101. Given a few pairs of names in the order child, father. The input is a person name and level number. The output should be the number of children in that particular level for the person given.
Example:
Input:
[
{Ram, Syam},
{Akil, Syam},
{Nikil, Ram},
{Subhash, Ram},
{Karthik, Akil}
];
Syam 2

Output: 3 (Syam has Ram and Akil in level 1 and in level 2 he have Nikil, Subhash, Karthik. So the answer is 3).

102. Given an array of positive integers. The output should be the number of occurrences of each number.
Example:
Input: {2, 3, 2, 6, 1, 6, 2}
Output:
1 – 1
2 – 3

3 – 1
6 – 2

103. Given an array, find the minimum of all the greater numbers for each element in the array.

Sample:
Array : {2, 3, 7, ¬1, 8, 5, 11}

Output:
{2¬>3, 3¬>5, 7¬>8, ¬1¬>2, 8¬>11, 5¬>7, 11¬>}

104. Find the largest sum contiguous subarray which should not have negative numbers. We have to print the sum and the corresponding array elements which brought up the sum.

Sample:
Array : {¬2, 7, 5, ¬1, 3, 2, 9, ¬7}

Output:
Sum : 14
Elements : 3, 2, 9

105. Given a string, we have to reverse the string without changing the position of punctuations and spaces.

Sample: house no : 123@ cbe
Output: ebc32 1o : nes@ uoh

106. Given a 2D grid of characters, you have to search for all the words in a dictionary by moving only along two directions, either right or down. Print the word if it occurs.

Sample :
a z o l
n x h o
v y i v
o r s e
Dictionary = {van, zoho, love, are, is}

Output:
zoho
love
Is

107. Given a string, change the order of words in the string (last string should come first).

Should use RECURSION

Sample: one two three
Output : three two one

108. Input
arr: 23, 34, 45, 66, 1, 12
target: 1

Output
Founded at the index of: 4
Explaination:
    1
66   12
45   23
    34
find the mid element of the array then check the mid with the start index value
arr[i] <= arr[mid]
    whether the mid value is greater than or equal to mid then, that part will be
[23,34,45,66] are one part of sorted element
or
if arr is [23,34,45,1,2,12] then [1,2,12] is other part of sorted element
then check the target value is under the sorted part of other
according to that change the starting and ending position.

109. INPUT:
Screen length = 20
Sentence = Today is wednesda

OUTPUT:
Today***is**wednesda

EXPLAINATION:
first we need to find the number of space in the sentence.
   remaining screen length should be find by subtracting the screenLength - sentence.length()
   find the how many space need to add by (remainingScreenLength)/ spaceCount
   when the addSpaceCount is odd the first space should add +1 than addSpaceValue and other place add normal.

110. Find the frequence of the array without collection and inbuilt method in java.

111. Write a program that will print the sum of diagonal elements of a 10X10 matrix. The program will take a total of 100 numbers as input (10 numbers will be input per line and each number will be separated by a space).

Example 1

Input: 1 2 3 4 5 6 7 8 9 0
0 1 2 3 4 5 6 7 8 0
3 4 5 6 7 8 9 6 4 0
2 3 4 5 6 7 8 9 3 2
3 4 5 6 7 4 3 2 1 3
3 4 5 6 2 4 4 2 4 6
2 3 4 6 2 4 6 2 3 5
2 3 5 6 2 4 6 2 3 5
2 4 6 2 1 4 3 3 5 2
3 3 5 2 4 6 2 1 4 6
Output: 42
Example 2

Input: 1 22 33 44 55 66 77 88 99 100
100 1 88 77 66 55 44 33 22 11
88 88 1 66 55 44 33 22 11 100
88 77 66 1 44 33 22 11 100 99
77 66 55 44 1 22 11 88 99 100
66 55 44 33 22 1 77 88 99 100
44 33 22 11 100 99 1 77 66 55
33 22 11 100 99 88 77 1 55 44
22 11 100 99 88 77 66 55 1 33
100 11 22 33 44 55 99 88 77 1
Output: 10

112. Input:6

Output
1
2 4
3 5 7
6 8 10 12
9 11 13 15 17
14 16 18 20 22 24

113. Input: 5

Output:
1 2 3 4 5
2 3 4 5 1
3 4 5 1 2

4 5 1 2 3
5 1 2 3 4

114. Input: 5

Output:
```
        1
      2 1
    3 2 1
  4 3 2 1
5 4 3 2 1
```

115. Integer to English Words
Convert a non-negative integer num to its English words representation.
Example 1:

Input: num = 123
Output: "One Hundred Twenty Three"
Example 2:

Input: num = 12345
Output: "Twelve Thousand Three Hundred Forty Five"
Example 3:

Input: num = 1234567
Output: "One Million Two Hundred Thirty Four Thousand Five Hundred Sixty Seven"

116. Remove the palindrome
Example 1:
Input: Malayalam is my mother tongue
Output: is my mother tongue

117. Compare Version Numbers

Given two version strings, version1 and version2, compare them. A version string consists of revisions separated by dots '.'. The value of the revision is its integer conversion ignoring leading zeros.

To compare version strings, compare their revision values in left-to-right order. If one of the version strings has fewer revisions, treat the missing revision values as 0.

Return the following:

If version1 < version2, return -1.

If version1 > version2, return 1.
Otherwise, return 0.

Example 1:

Input: version1 = "1.2", version2 = "1.10"

Output: -1

Explanation:

version1's second revision is "2" and version2's second revision is "10": 2 < 10, so version1 < version2.

Example 2:

Input: version1 = "1.01", version2 = "1.001"

Output: 0

Explanation:

Ignoring leading zeroes, both "01" and "001" represent the same integer "1".

ZOHO Round 3 Questions

1. Super Store

Description:
Build an online shopping interface where buyers and sellers can engage in efficient transactions across a wide range of products.

Modules:
- **Profile Service**
- **Inventory Service**
- **Order Service**
- **Payment Service**

Requirements:
- Allow buyers and sellers to register and log in.
- Allow sellers to manage their inventory of products.

  - Operations: `addItem()`, `updateItem()`
- Allow buyers to add items to the cart, place orders, and make payments.
  - Operations: `listInventory()`, `buyItem()`, `addToCart()`, `makePayment()`
- Validate orders against inventory.
- Password validation and encryption.

---

2. Call Taxi Booking Application

Description:
Design a taxi booking application where customers can book taxis available at certain points in a linear route.

Problem Assumptions:
- **Taxi Count:** Assume 4 taxis for simplicity, but it should scale to any number of taxis.
- **Points on Route:** A, B, C, D, E, F (each 15 km apart)
- **Travel Time Between Points:** 60 minutes
- **Charges:**
  - Minimum Rs.100 for the first 5 km
  - Rs.10 per km thereafter
- **Initial Taxi Position:** All taxis are stationed at A.

Booking Rules:
- When a customer books a taxi:
  - A free taxi at the pickup point is allocated.
  - If no free taxi is available, the nearest taxi is allocated.
  - If two taxis are free at the same point, the one with lower earnings is allocated.
  - Taxis only charge from the pickup point to the drop point.
  - If no taxi is available, the booking is rejected.

Modules:
1. **Call Taxi Booking**

   Sample Inputs and Outputs:

   Input 1:
   - Customer ID: 1
   - Pickup Point: A
   - Drop Point: B
   - Pickup Time: 9

   Output 1:
   - Taxi can be allotted.

- Taxi-1 is allotted.

Input 2:
- Customer ID: 2
- Pickup Point: B
- Drop Point: D
- Pickup Time: 9

Output 2:
- Taxi can be allotted.
- Taxi-2 is allotted.

Input 3:
- Customer ID: 3
- Pickup Point: B
- Drop Point: C
- Pickup Time: 12

Output 3:
- Taxi can be allotted.
- Taxi-1 is allotted.

2. **Display Taxi Details**

Output Example:
```
Taxi-1  Total Earnings: Rs. 400
BookingID  CustomerID  From  To  PickupTime  DropTime  Amount
1          1           A     B   9           10        200
3          3           B     C   12          13        200

Taxi-2  Total Earnings: Rs. 350
BookingID  CustomerID  From  To  PickupTime  DropTime  Amount
2          2           B     D   9           11        350
```

---

3. Railway Reservation System

Description:
Develop a railway reservation application with the following types of seats:
- **AC Coach**
- **Non-AC Coach**

- **Seater**

Each seat type has a capacity of 60 seats with a waiting list max of 10 seats.

Modules:
1. **Booking**
2. **Availability Checking**
3. **Cancellation**
4. **Prepare Chart**

---

4. Matrix Game with Ray Rules

Description:
A matrix-based game where rays interact with atoms inside a grid according to specific rules.

Problem Setup:
- **Grid:**
  ```
  R3  | -   -   - |
  R2  | -   -   - |
  R1  | -   -   - |
        C1  C2  C3
  ```
- **Rules:**
  - **Rule 1:** A ray with an atom in its path prints 'H' (Hit).
  - **Rule 2 & 3:** A ray with an atom in a diagonal adjacent position refracts.
  - **Rule 4:** A ray with atoms in both diagonal positions reflects.
  - **Rule 5:** Deflection happens in the order of input rays.

Sample Input and Output:

Input:
- Position of atoms and the rays originating from outside the box.
- Example:
  ```
  3
  3 1
  2 2
  1 3
  3
  R3 C1 C3
  ```

Output:
- Display the final state of the box.

Sample Matrix Display:
```
    C1  H   C3
 R3  | -   -   - | R3
 H   | -   X   - | H
 R1  | -   -   - | R1
     C1  H   C3
```

Here's a refined version of the question based on your description:

---

5. Brick Breaker

1. **Array Structure:**
   You have a 7x7 grid array representing a game where:
   - `w` represents walls surrounding the area.
   - `g` represents ground.
   - `o` represents the ball.
   - Numbers (`1`, `2`, `3`, etc.) represent bricks with each number indicating the strength of the brick (i.e., how many hits it can take before disappearing).

2. **Initial Setup:**
   - **Input size**: 7
   - **Number of bricks**: 6
   - **Positions of bricks**: (2, 2), (2, 3), (2, 4), (3, 2), (3, 3), (3, 4)

3. **Game Mechanics:**
   - The ball starts with 5 lives.
   - The ball can move in three directions:
     - **St (Straight)**: The ball moves vertically upwards.
     - **Lt (Left Diagonal)**: The ball moves diagonally left-upward.
     - **Rt (Right Diagonal)**: The ball moves diagonally right-upward.
   - When the ball hits a brick, the brick's strength decreases by 1. If a brick’s strength reaches 0, it disappears.
   - After hitting a brick, the ball returns to its original position unless the brick disappears, in which case it moves into the empty space left by the brick.

4. **Commands:**

- **St (Straight)**: If there is a brick in the ball's path, it hits it, decreasing the brick’s strength. The ball then returns to its starting position if the brick remains. If the brick disappears (reaches 0 strength), the ball moves into the position where the brick was.
- **Lt (Left Diagonal)**: The ball moves diagonally left. If it encounters a brick, it hits it, reducing the brick's strength by 1. If the brick disappears, the ball takes its position and continues moving downward.
- **Rt (Right Diagonal)**: The ball moves diagonally right, following the same rules as Lt.

5. **Example Gameplay:**
- **Input Grid:**

```
w w w w w w w
w           w
w   1 1 1   w
w   1 1 1   w
w           w
w           w
w g g o g g w
```

- **Example Commands and Output:**
  - **Input**: `St`
    - **Output**:

```
w w w w w w w
w           w
w   1 1 1   w
w   1 1 1   w
w           w
w           w
w g g o g g w
Ball count: 5
```

Explanation: The ball moved straight up but encountered no bricks, so it returned to its initial position.

  - **Input**: `lt`
    - **Output**:

```
w w w w w w w
```

```
w         w
w   1 1 1   w
w   1 1     w
w         w
w         w
w g o g g g w
Ball count: 4
```

Explanation: The ball moved diagonally left. Since there was no brick immediately in the way, it hit the wall and reflected horizontally, encountering a brick at (3, 2). After hitting the brick, the ball moved downward, causing the brick to disappear. The ball count is now reduced by 1.

6. **Advanced Modules:**
   - In later modules of the game:
     - Each brick has a value indicating its strength. Every hit reduces the strength by 1, and the brick disappears only when its strength reaches 0.
     - The game consists of 7 modules, each adding new mechanics or rules to enhance gameplay.